Université Abdelmalek Essaadi

Faculté des Sciences et Techniques de Tanger

LICENCE |IDAI — Back end (Python) | 2025/2026

RAPPORT TECHNIQUE

Projet de Fin de Module

Système de Gestion Scolaire « PreSkool »

Realisé par : Monssif Elkadi

Formation Licence IDAI

Module Développement Web Avancé — Back end (Python)

Encadrant Prof. Sara AHSAIN

Sommaire

1. Contexte et Présentation du Projet
2. Objectifs de l'Application
3. Architecture Technique
4. Diagramme de Classes UML
5. Choix Techniques
6. Description des Modules
7. Fonctionnalités CRUD — Détail Complet
8. Système de Rôles et Sécurité
9. Captures d'Écran
10. Difficultés Rencontrées
11. Améliorations Possibles
12. Conclusion

1. Contexte et Présentation du Projet

1.1 L'École Fictive — PreSkool Academy

PreSkool Academy est un établissement d'enseignement supérieur fictif basé à Tanger. Comme beaucoup d'institutions de taille moyenne, elle gère plusieurs centaines d'étudiants, une équipe d'enseignants répartis dans différents départements, et un personnel administratif qui s'occupe de tout le reste.

Le problème concret qu'on a voulu résoudre : toute la gestion — étudiants, emplois du temps, examens, absences — se faisait manuellement, avec des fichiers Excel éparpillés et des dossiers papier. Ce projet est la réponse numérique à ce problème.

1.2 Ce que PreSkool apporte concrètement

Avant PreSkool

Gestion manuelle et fragmentée : fichiers Excel, dossiers papier, erreurs fréquentes, aucune visibilité globale pour l'administration.

Après PreSkool

Plateforme web centralisée : chaque acteur (admin, enseignant, étudiant) accède à ce qui le concerne, depuis n'importe quel navigateur.

2. Objectifs de l'Application

L'objectif principal était de construire une application web Django complète, avec des opérations CRUD pour chaque entité, et un système d'accès différencié selon le rôle de l'utilisateur. Voici les fonctionnalités couvertes :

Module	Ce que ça fait	Qui y a accès
Étudiants	Ajouter, consulter, modifier, supprimer les fiches étudiants et parents	Admin, Teacher
Enseignants	Gérer les fiches enseignants avec photo et département	Admin
Départements	Structurer l'école en départements pédagogiques	Admin
Matières	Lier chaque matière à un département et un enseignant	Admin, Teacher
Emploi du temps	Planifier les créneaux par classe, jour et horaire	Admin, Teacher

Examens	Créer des examens et saisir les notes sur 20	Admin, Teacher
Jours fériés	Gérer le calendrier des congés scolaires	Admin
Rôles & Sécurité	Accès différenciés avec décorateurs Python	Tous

3. Architecture Technique

3.1 Pourquoi Django et le pattern MVT

Django a été choisi parce qu'il embarque nativement tout ce dont on a besoin : un ORM pour parler à la base de données sans écrire du SQL brut, un système d'authentification prêt à l'emploi, et une interface d'administration automatique. Le pattern MVT qu'il impose force naturellement à bien séparer la logique métier de l'affichage, ce qui rend le code plus lisible et maintenable.

Couche	Fichier	Rôle
Model (M)	models.py	Définit les tables en base de données via des classes Python
View (V)	views.py	Traite les requêtes, applique la logique, retourne une réponse
Template (T)	templates/*.html	Génère le HTML final avec les données injectées par la vue

3.2 Structure des Dossiers

Le projet est découpé en 9 applications Django indépendantes, chacune responsable d'un seul domaine. Cette organisation facilite la maintenance et permet d'ajouter de nouvelles fonctionnalités sans toucher au reste.

```
pfm-school-management/
├── monenv/              ← Environnement virtuel Python (isolé)
└── school/              ← Répertoire principal du projet Django
    ├── manage.py        ← Point d'entrée Django (commandes)
    ├── requirements.txt ← Dépendances du projet
    ├── db.sqlite3       ← Base de données SQLite
    │
    ├── school/          ← Configuration centrale
    │   ├── settings.py  ← INSTALLED_APPS, AUTH_USER_MODEL, STATIC...
    │   ├── urls.py      ← Routage principal (inclut les apps)
    │   └── wsgi.py
    │
    ├── faculty/         ← Dashboards (admin / teacher / student)
    ├── home_auth/       ← Auth + CustomUser + Décorateurs RBAC
    ├── student/         ← CRUD Étudiants + Parents
    ├── teacher/         ← CRUD Enseignants
    ├── department/      ← CRUD Départements
```

```
├── subject/            ← CRUD Matières
├── holiday/            ← CRUD Jours Fériés
├── timetable/          ← CRUD Emploi du Temps + Bonus iframe
├── exam/               ← CRUD Examens + Résultats avec note /20
├── Rapport/
├── Video-demonstratif/
├── static/assets/      ← CSS, JS, images Bootstrap (PreSkool theme)
├── media/              ← Photos uploadées (students/, teachers/)
└── templates/
    ├── Home/           ← base.html + dashboards
    ├── authentication/ ← login, register, logout, reset password
    ├── students/       ← templates CRUD étudiants
    ├── teachers/       ← templates CRUD enseignants
    ├── departments/
    ├── subjects/
    ├── holidays/
    ├── timetable/
    └── exams/
```

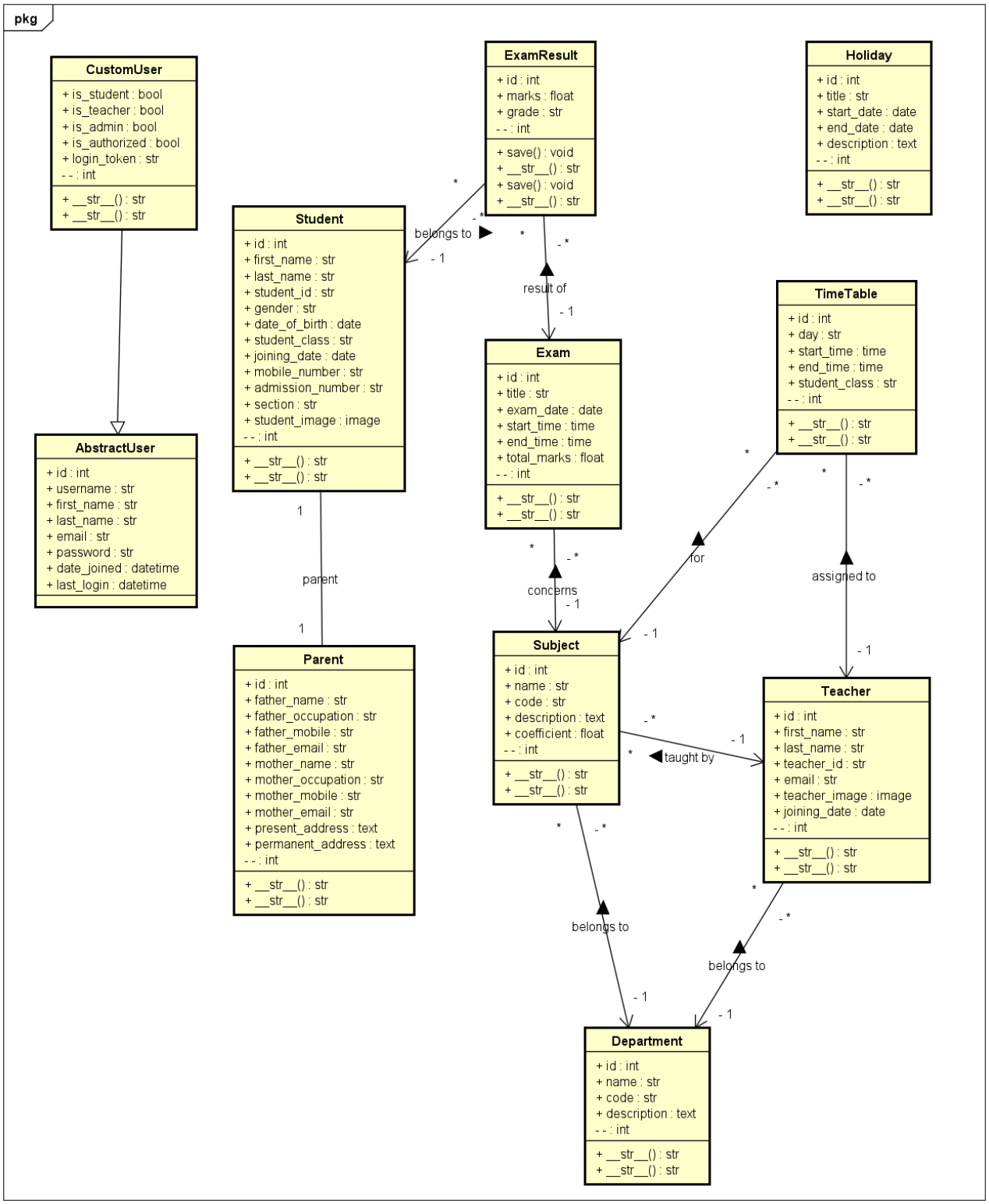
3.3 Flux d'une Requête HTTP

Voici ce qui se passe concrètement quand un utilisateur clique sur un lien dans l'application :

Étape	Qui intervient	Ce qui se passe
1	Navigateur	Envoie une requête HTTP vers Django
2	school/urls.py	Identifie l'app concernée et délègue l'URL
3	app/urls.py	Appelle la vue correspondante
4	Décorateur	Vérifie que l'utilisateur est connecté et a le bon rôle
5	views.py	Traite la logique et interroge la base via le modèle
6	models.py	Retourne les données demandées
7	template.html	Génère le HTML avec les données reçues
8	Navigateur	Affiche la page finale à l'utilisateur

4. Diagramme de Classes UML

Le diagramme ci-dessous représente l'ensemble des entités de l'application et leurs relations. Il a été conçu pour refléter exactement les modèles Django implémentés dans le projet.



4.1 Relations entre les Entités

Chaque relation correspond à un champ Django dans le modèle correspondant :

Entité	Champ Django	Liée à	Signification
Student	OneToOneField	Parent	Un étudiant a exactement un dossier parent
Teacher	ForeignKey	Department	Plusieurs enseignants peuvent appartenir au même département
Subject	ForeignKey	Department	Une matière appartient à un seul département
Subject	ForeignKey	Teacher	Une matière est assignée à un enseignant
TimeTable	ForeignKey	Subject	Un créneau est associé à une matière précise
TimeTable	ForeignKey	Teacher	Un créneau est assuré par un enseignant
Exam	ForeignKey	Subject	Un examen porte sur une matière
ExamResult	ForeignKey	Exam	Un résultat est rattaché à un examen
ExamResult	ForeignKey	Student	Un résultat appartient à un étudiant

5. Choix Techniques

Les technologies utilisées ont été choisies pour leur simplicité de mise en place et leur adéquation avec les besoins du projet. L'objectif était d'aller à l'essentiel sans sur-engineering.

Technologie	Version	Pourquoi ce choix
Python	3.13	Langage imposé par le module — lisible et adapté au développement web backend
Django	5.x	ORM intégré, admin auto, système d'auth natif — tout ce qu'il faut sans configuration excessive
SQLite	—	Aucune configuration requise, idéal pour un projet de développement — la base est un simple fichier
Pillow	Latest	Requis obligatoirement par Django dès qu'on utilise un champ ImageField dans un modèle
Bootstrap 4	4.x	Template PreSkool fourni par le professeur — design responsive prêt à l'emploi
HTML/CSS/JS	—	Frontend rendu côté serveur via les templates Django

6. Description des Modules

6.1 home_auth — Authentification & Gestion des Rôles

C'est le module le plus critique de l'application. Sans lui, rien ne fonctionne. Il gère la création des comptes utilisateurs avec un modèle personnalisé qui étend AbstractUser de Django, en ajoutant trois champs booléens pour les rôles.

Modèle CustomUser

```
class CustomUser(AbstractUser):
    username = models.CharField(max_length=100, unique=True)
    email = models.EmailField(max_length=255, unique=True, db_index=True)
    is_authorized = models.BooleanField(default=False)
    login_token = models.CharField(max_length=6, blank=True, null=True)
    first_name = models.CharField(max_length=30, blank=True)
    last_name = models.CharField(max_length=30, blank=True)
    date_joined = models.DateTimeField(auto_now_add=True)
    is_student = models.BooleanField(default=False)
    is_admin = models.BooleanField(default=False)
    is_teacher = models.BooleanField(default=False)
```

Le fait d'étendre AbstractUser plutôt que de repartir de zéro permet de garder toutes les fonctionnalités natives de Django (gestion des mots de passe, sessions, permissions) tout en ajoutant notre logique de rôles.

6.2 faculty — Dashboards par Rôle

Ce module gère uniquement les dashboards. Sa vue principale index() détecte le rôle de l'utilisateur connecté et le redirige vers le tableau de bord approprié.

```
def index(request):
    if request.user.is_admin or request.user.is_superuser:
        context = {
            'student_count': Student.objects.count(),
            'teacher_count': Teacher.objects.count(),
            'department_count': Department.objects.count(),
            'subject_count': Subject.objects.count(),
        }
        return render(request, 'Home/admin_dashboard.html', context)
```

```
    elif request.user.is_teacher:
        today = timezone.localdate()
        end_date = today + timedelta(days=30)
        upcoming_holidays = Holiday.objects.filter(
            date__range=(today, end_date)
        ).order_by('date')
        teacher_profile = (
            Teacher.objects.filter(email=request.user.email).first()
            or Teacher.objects.filter(
                first_name=request.user.first_name,
                last_name=request.user.last_name,
            ).first()
        )
        context = {
            'teacher_profile': teacher_profile,
            'upcoming_holidays': upcoming_holidays,
            'subjects': Subject.objects.all()[:5],
            'timetables': TimeTable.objects.all()[:5]
        }
        return render(request, 'Home/teacher_dashboard.html', context)
```

6.3 student — Étudiants & Parents

Ce module utilise deux modèles liés par une relation OneToOne : chaque étudiant a exactement un dossier parent. Cette structure a été imposée par le sujet du TP et reflète bien la réalité.

```
class Parent(models.Model):
    father_name       = models.CharField(max_length=100)
    father_occupation = models.CharField(max_length=100, blank=True)
    father_mobile     = models.CharField(max_length=15)
    father_email      = models.EmailField(max_length=100)
    mother_name       = models.CharField(max_length=100)
    mother_occupation = models.CharField(max_length=100, blank=True)
    mother_mobile     = models.CharField(max_length=15)
    mother_email      = models.EmailField(max_length=100)
    present_address   = models.TextField()
    permanent_address = models.TextField()

class Student(models.Model):
    first_name       = models.CharField(max_length=100)
    last_name        = models.CharField(max_length=100)
    student_id       = models.CharField(max_length=20)
    gender           = models.CharField(max_length=10,
choices=[('Male','Male'),('Female','Female')])
    date_of_birth    = models.DateField()
    student_class    = models.CharField(max_length=50)
    joining_date     = models.DateField()
```

```
    mobile_number     = models.CharField(max_length=15)
    admission_number = models.CharField(max_length=20)
    section           = models.CharField(max_length=10)
    student_image     = models.ImageField(upload_to='students/', blank=True)
    parent            = models.OneToOneField(Parent, on_delete=models.CASCADE)
```

6.4 teacher — Enseignants

```
class Teacher(models.Model):
    first_name     = models.CharField(max_length=100)
    last_name      = models.CharField(max_length=100)
    teacher_id     = models.CharField(max_length=20, unique=True)
    gender         = models.CharField(max_length=10,
choices=[('Male','Male'),('Female','Female')])
    date_of_birth = models.DateField()
    email          = models.EmailField(unique=True)
    mobile_number = models.CharField(max_length=15)
    address        = models.TextField(blank=True)
    department     = models.ForeignKey(Department, on_delete=models.SET_NULL, null=True,
blank=True)
    teacher_image = models.ImageField(upload_to='teachers/', blank=True)
    joining_date   = models.DateField()
```

6.5 department — Départements

```
class Department(models.Model):
    name = models.CharField(max_length=100 , unique=True)
    description = models.TextField(blank=True)
```

6.6 subject — Matières

```
class Subject(models.Model):
    name           = models.CharField(max_length=100)
    subject_code = models.CharField(max_length=20, unique=True)
    department     = models.ForeignKey(Department, on_delete=models.CASCADE)
    teacher        = models.ForeignKey(Teacher, on_delete=models.SET_NULL, null=True,
blank=True)
```

6.7 holiday — Jours Fériés

```
class Holiday(models.Model):
    name          = models.CharField(max_length=100)
    date          = models.DateField()
    description = models.TextField(blank=True)
```

6.8 timetable — Emploi du Temps

```
DAYS = [
    ('Monday', 'Monday'),
    ('Tuesday', 'Tuesday'),
    ('Wednesday', 'Wednesday'),
    ('Thursday', 'Thursday'),
    ('Friday', 'Friday'),
    ('Saturday', 'Saturday'),
]
class TimeTable(models.Model):
    day           = models.CharField(max_length=10, choices=DAYS)
    start_time    = models.TimeField()
    end_time      = models.TimeField()
    subject       = models.ForeignKey(Subject, on_delete=models.CASCADE)
    teacher       = models.ForeignKey(Teacher, on_delete=models.SET_NULL,
null=True, blank=True)
    student_class = models.CharField(max_length=50)
```

6.9 exam — Examens & Résultats

Ce module gère deux entités : l'examen lui-même, et les résultats individuels par étudiant. La mention est calculée automatiquement lors de la sauvegarde, selon le système de notation marocain sur 20.

```
class Exam(models.Model):
    name          = models.CharField(max_length=100)
    subject       = models.ForeignKey(Subject, on_delete=models.CASCADE)
    date          = models.DateField()
    start_time    = models.TimeField()
    student_class = models.CharField(max_length=50)

class ExamResult(models.Model):
    exam           = models.ForeignKey(Exam, on_delete=models.CASCADE)
    student        = models.ForeignKey(Student, on_delete=models.CASCADE)
    marks          = models.FloatField()
    grade          = models.CharField(max_length=5, blank=True)

    class Meta:
        unique_together = ('exam', 'student')

    def save(self, *args, **kwargs):
```

7. Fonctionnalités CRUD — Détail Complet

Le CRUD (Create, Read, Update, Delete) est au cœur du projet. C'est ce qui permet à chaque module de fonctionner. Voici le détail complet pour chaque entité : les URLs, les méthodes HTTP utilisées, et les accès autorisés.

7.1 Vue d'ensemble CRUD

Entité	Lister	Ajouter	Modifier	Supprimer	Accès
Students	/student/	/student/add/	/student/edit/<id>/	/student/delete/<id>/	Admin, Teacher
Teachers	/teacher/	/teacher/add/	/teacher/edit/<id>/	/teacher/delete/<id>/	Admin
Departments	/department/	/department/add/	/department/edit/<id>/	/department/delete/<id>/	Admin
Subjects	/subject/	/subject/add/	/subject/edit/<id>/	/subject/delete/<id>/	Admin, Teacher
Holidays	/holiday/	/holiday/add/	/holiday/edit/<id>/	/holiday/delete/<id>/	Admin
Timetable	/timetable/	/timetable/add/	/timetable/edit/<id>/	/timetable/delete/<id>/	Admin, Teacher
Exams	/exam/	/exam/add/	/exam/edit/<id>/	/exam/delete/<id>/	Admin, Teacher

7.2 Pattern d'une Vue CRUD — Exemple avec Student

Chaque vue CRUD suit le même pattern : si la requête est GET, on affiche le formulaire vide. Si c'est POST, on récupère les données et on les sauvegarde en base.

Ajouter un étudiant — add_student (views.py)

```
def add_student(request):
    if request.method == 'POST':
        # Create Parent first
        parent = Parent.objects.create(
            father_name=request.POST['father_name'],
            father_occupation=request.POST['father_occupation'],
            father_mobile=request.POST['father_mobile'],
            father_email=request.POST['father_email'],
```

```
            mother_name=request.POST['mother_name'],
            mother_occupation=request.POST['mother_occupation'],
            mother_mobile=request.POST['mother_mobile'],
            mother_email=request.POST['mother_email'],
            present_address=request.POST['present_address'],
            permanent_address=request.POST['permanent_address'],
        )
        student = Student(
            first_name=request.POST['first_name'],
            last_name=request.POST['last_name'],
            student_id=request.POST['student_id'],
            gender=request.POST['gender'],
            date_of_birth=request.POST['date_of_birth'],
            student_class=request.POST['student_class'],
            joining_date=request.POST['joining_date'],
            mobile_number=request.POST['mobile_number'],
            admission_number=request.POST['admission_number'],
            section=request.POST['section'],
            parent=parent,
        )
        if 'student_image' in request.FILES:
            student.student_image = request.FILES['student_image']
        student.save()
        return redirect('student_list')
    return render(request, 'students/add-student.html')
```

Modifier un étudiant — edit_student (views.py)

```
@teacher_required
def edit_student(request, student_id):
    student = get_object_or_404(Student, student_id=student_id)
    if request.method == 'POST':
        student.first_name = request.POST['first_name']
        student.last_name = request.POST['last_name']
        student.gender = request.POST['gender']
        student.date_of_birth = request.POST['date_of_birth']
        student.student_class = request.POST['student_class']
        student.joining_date = request.POST['joining_date']
        student.mobile_number = request.POST['mobile_number']
        student.admission_number = request.POST['admission_number']
        student.section = request.POST['section']
        if 'student_image' in request.FILES:
            student.student_image = request.FILES['student_image']
        student.save()
        # Update parent info
        student.parent.father_name = request.POST['father_name']
        student.parent.father_occupation = request.POST['father_occupation']
        student.parent.father_mobile = request.POST['father_mobile']
        student.parent.father_email = request.POST['father_email']
```

```
        student.parent.mother_name = request.POST['mother_name']
        student.parent.mother_occupation = request.POST['mother_occupation']
        student.parent.mother_mobile = request.POST['mother_mobile']
        student.parent.mother_email = request.POST['mother_email']
        student.parent.present_address = request.POST['present_address']
        student.parent.permanent_address = request.POST['permanent_address']
        student.parent.save()
        return redirect('student_list')
    return render(request, 'students/edit-student.html', {'student': student})
```

Supprimer — delete_student (views.py)

```
@teacher_required
def delete_student(request, student_id):
    student = get_object_or_404(Student, student_id=student_id)
    if request.method == 'POST':
        student.parent.delete()
    return redirect('student_list')
```

7.3 Exemple de Template CRUD — Liste des Étudiants

Les templates utilisent les balises Django pour afficher les données dynamiquement et lier les boutons d'action aux bonnes URLs.

```
<tbody>
{% for student in student_list %}
<tr>
 <td>{{ student.student_id }}</td>
<td>
 <h2 class="table-avatar">
 <a href="{% url 'view_student' student.student_id %}"class="avatar avatar-sm mr-2">
 {% if student.student_image %}
<img class="avatar-img rounded-circle" src="{{ student.student_image.url }}" alt="{{
student.first_name }}">
{% else %}
 <img class="avatar-img rounded-circle" src="{% static 'assets/img/profiles/avatar-01.jpg' %}"
alt="No Image">
{% endif %}
 </a>
<a href="{% url 'view_student' student.student_id %}">{{ student.first_name }} {{
student.last_name }}</a>
</h2>
 </td>
<td>{{ student.student_class }}</td>
<td>{{ student.date_of_birth|date:"d M Y" }}</td>
<td>{{ student.parent.father_name }} / {{ student.parent.mother_name }}</td>
<td>{{ student.mobile_number }}</td>
<td>{{ student.parent.present_address }}</td>
```

```
{% if request.user.is_admin or request.user.is_teacher %}
 <td class="text-right">
 <div class="actions">
<a href="{% url 'edit_student' student.student_id %}"
   class="btn btn-sm bg-success-light mr-2">
 <i class="fas fa-pen"></i>
 </a>
 <form action="{% url 'delete_student' student.student_id %}"
        method="POST" style="display:inline;">
{% csrf_token %}
 <button type="submit" class="btn btn-sm bg-danger-light"
         onclick="return confirm('Delete this student?');">
<i class="fas fa-trash"></i>
</button>
</form>
  </div>
</td>
{% endif %}
 </tr> {% empty %}
td colspan="8" class="text-center">No Students Found</td></tr>
{% endfor %}
```

7.4 URLs — Récapitulatif par App

Chaque application Django a son propre fichier urls.py. Voici comment les routes sont organisées, en prenant l'exemple de l'app teacher :

```
from django.urls import path
from . import views

urlpatterns = [
    path('', views.teacher_list, name='teacher_list'),
    path('add/', views.add_teacher, name='add_teacher'),
    path('view/<int:pk>/', views.view_teacher, name='view_teacher'),
    path('edit/<int:pk>/', views.edit_teacher, name='edit_teacher'),
    path('delete/<int:pk>/', views.delete_teacher, name='delete_teacher'),
]
```

8. Système de Rôles et Sécurité

La sécurité est implémentée à deux niveaux : côté serveur via des décorateurs Python, et côté interface via des conditions dans les templates. L'idée est qu'un utilisateur ne doit jamais pouvoir accéder à quelque chose qu'il n'est pas censé voir, même en tapant l'URL directement dans le navigateur.

8.1 Décorateurs Python — home_auth/decorators.py

Plutôt que de répéter les mêmes vérifications dans chaque vue, on a centralisé la logique dans un fichier dédié avec 3 décorateurs réutilisables :

```
from django.shortcuts import redirect

def admin_required(view_func):
    def wrapper(request, *args, **kwargs):
        if not request.user.is_authenticated:
            return redirect('login')
        if not request.user.is_admin and not request.user.is_superuser:
            return redirect('dashboard')
        return view_func(request, *args, **kwargs)
    return wrapper

def teacher_required(view_func):
.. .

def login_required_custom(view_func):
....
```

8.2 Application sur les Vues

```
# Exemple : teacher/views.py
from home_auth.decorators import admin_required

@admin_required          # ← Un seul décorateur suffit
def add_teacher(request):
    ...

# Exemple : subject/views.py
from home_auth.decorators import teacher_required

@teacher_required        # ← Admin ET Teacher peuvent accéder
def subject_list(request):
    ...
```

8.3 Tableau des Accès

Fonctionnalité	Admin	Teacher	Student
Admin Dashboard	✓	✗	✗
Teacher Dashboard	✗	✓	✗
Student Dashboard	✗	✗	✓
Students — CRUD	✓	✓	✗
Teachers — CRUD	✓	✗	✗
Departments — CRUD	✓	✗	✗
Subjects — CRUD	✓	✓	✗
Timetable — lecture	✓	✓	✓
Timetable — modif	✓	✓	✗
Exams — lecture	✓	✓	✓
Exams — modif	✓	✓	✗
Holidays — lecture	✓	✓	✓
Holidays — modif	✓	✗	✗

8.4 Comptes de Test

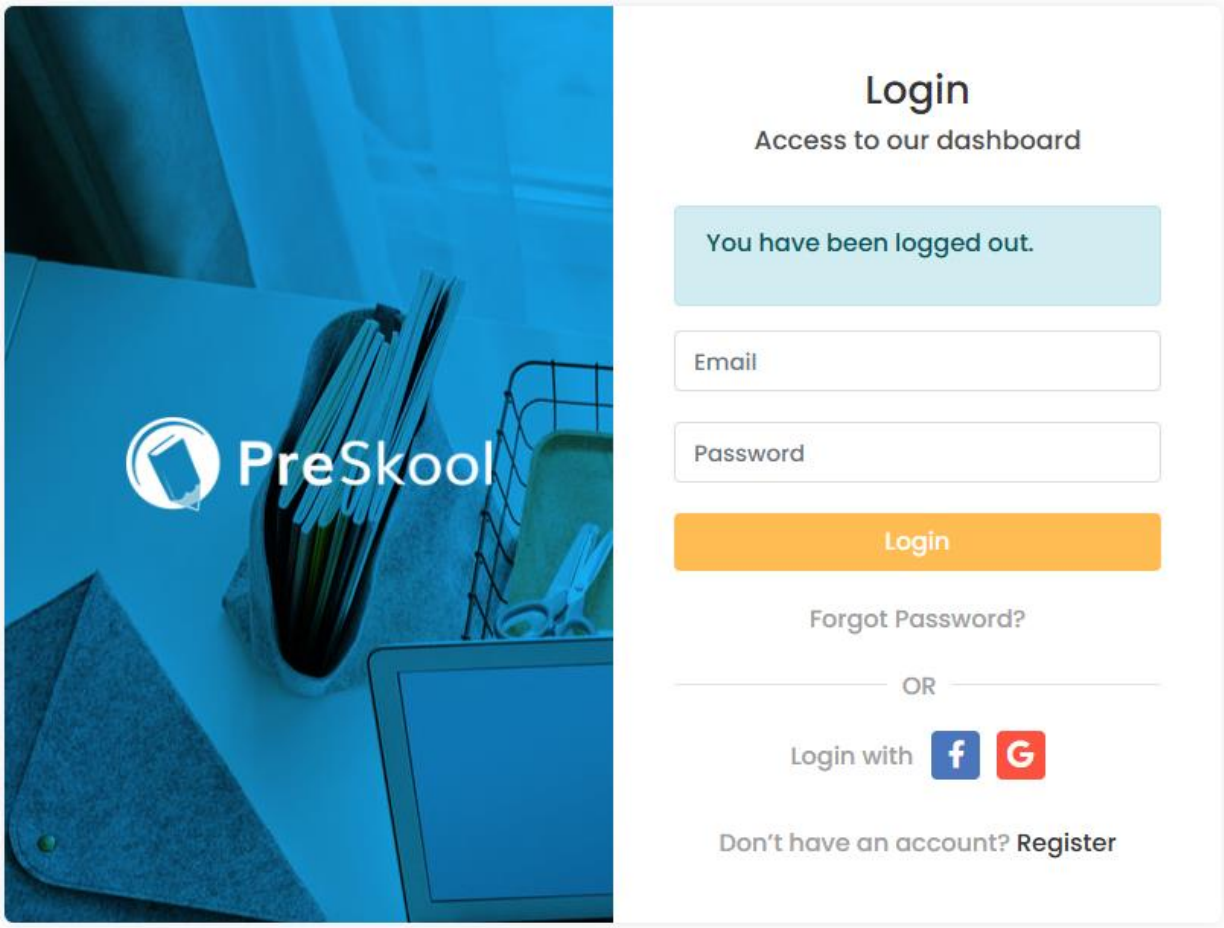
Rôle	E-mail	Mot de passe
Admin	admin@admin.com	admin123
Enseignant	teacher@preskool.com	teacher123
Étudiant	student@preskool.com	student123

9. Captures d'Écran

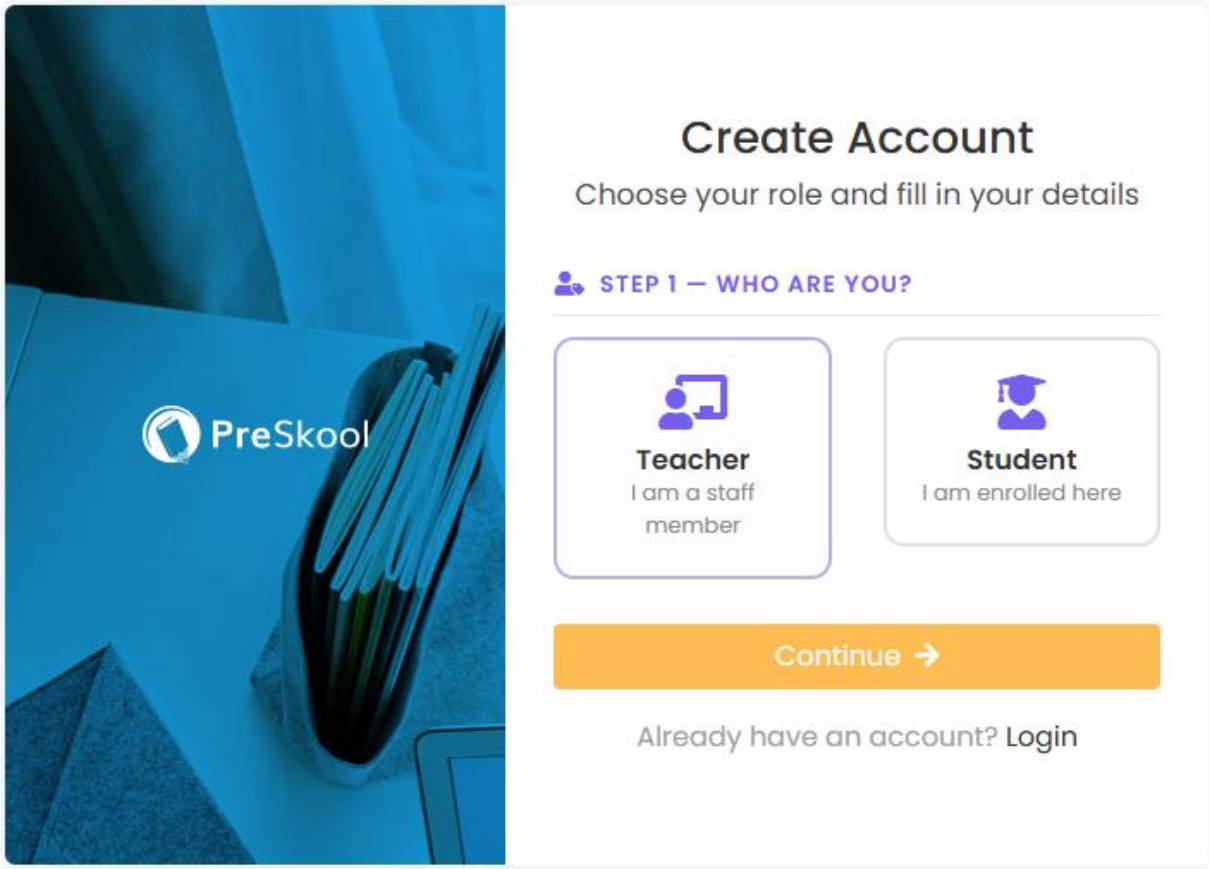
Les captures suivantes illustrent les principales fonctionnalités de l'application pour chaque type d'utilisateur.

9.1 Authentification :

Page de Connexion — Login

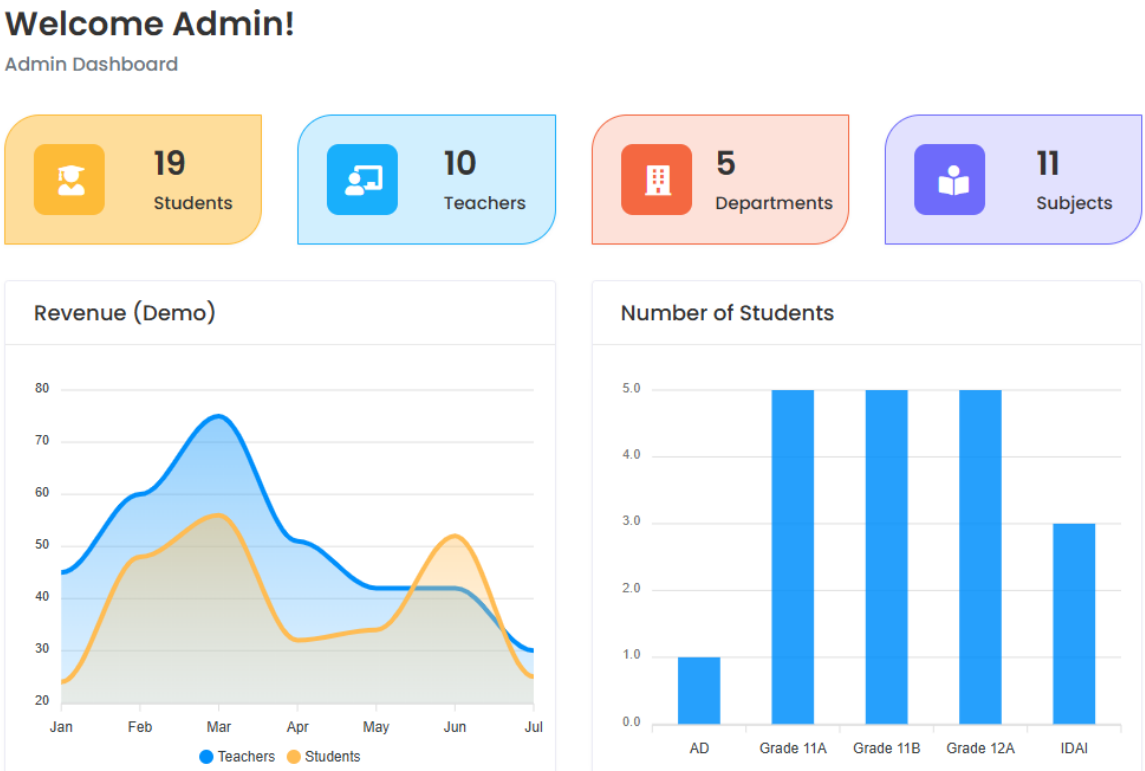


Page d'Inscription — Register avec choix du rôle :



9.2 Dashboards

Dashboard Admin — Statistiques globales (étudiants, profs, matières...) :

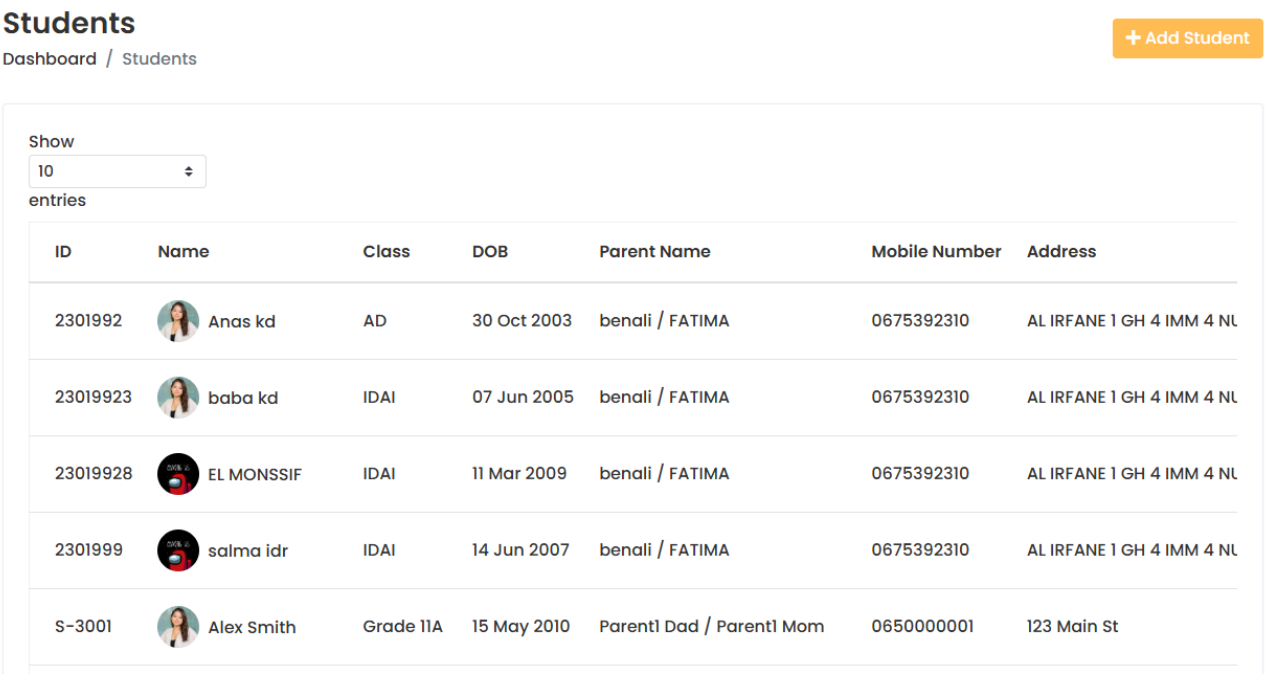


Dashboard Enseignant — Mes matières + Mon emploi du temps :

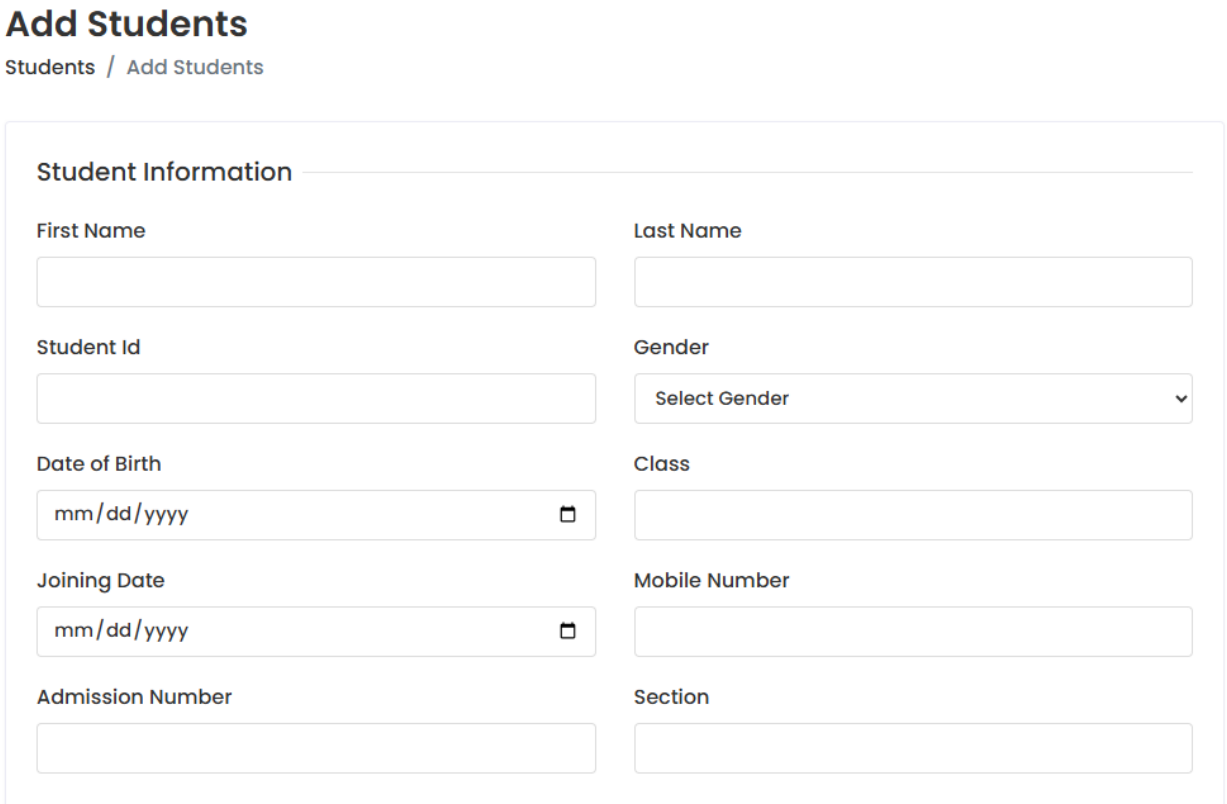
Dashboard Étudiant — Mes notes + Accès rapide :

9.3 CRUD Étudiants :

Liste des Étudiants — Vue tableau avec actions (Voir) :



Formulaire Ajout Étudiant — Informations étudiant + parent :



Formulaire Ajout Étudiant — Informations étudiant + parent :

Student Image

Choose File No file chosen

Parent Information

Father's Name

Father's Occupation

Father's Mobile

Father's Email

Mother's Name

Mother's Occupation

Mother's Mobile

Mother's Email

Present Address

Permanent Address

Submit

Formulaire Modification Étudiant — Champs pré-remplis :

Edit Student

Students / Edit Student

Student Information

First Name: baba

Last Name: kd

Student ID: 23019923

Gender: Male

Date of Birth: 06/07/2005

Class: IDAI

Joining Date: 03/02/2026

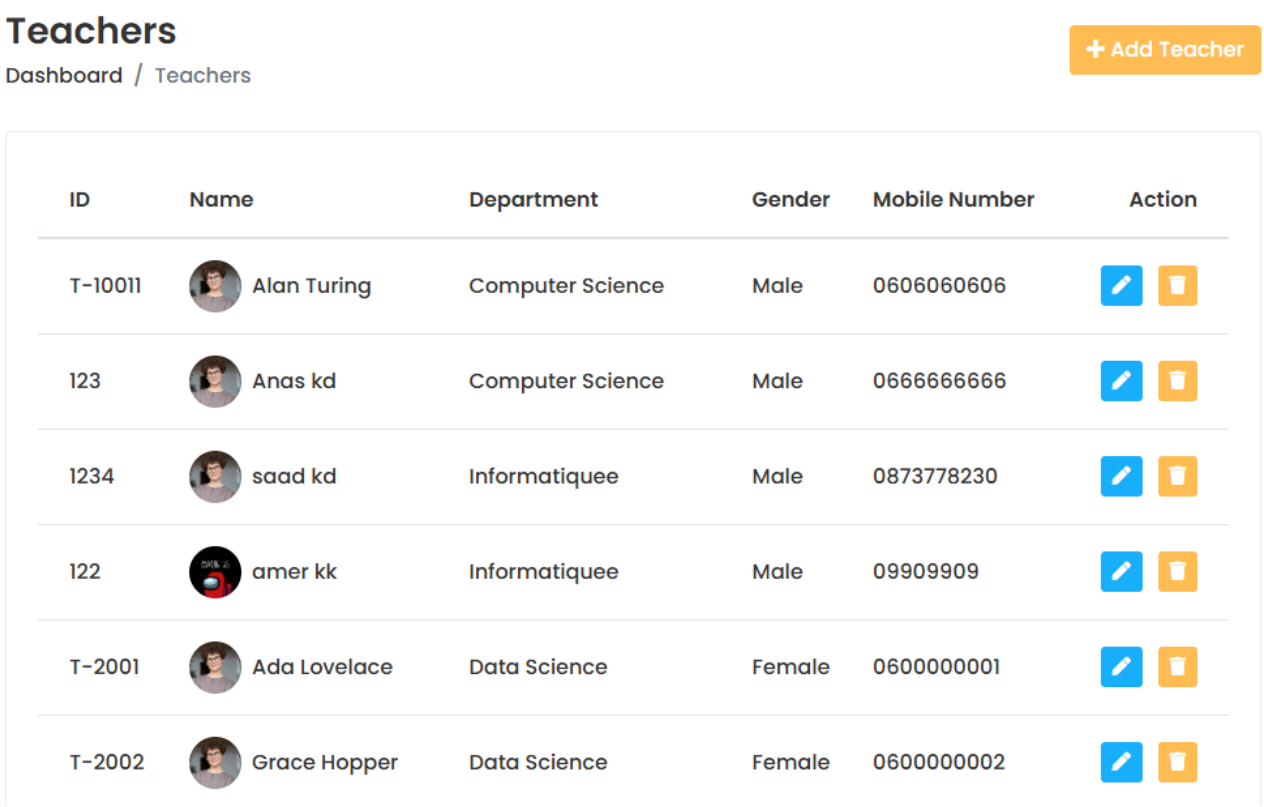
Mobile Number: 0675392310

Admission Number: 121211

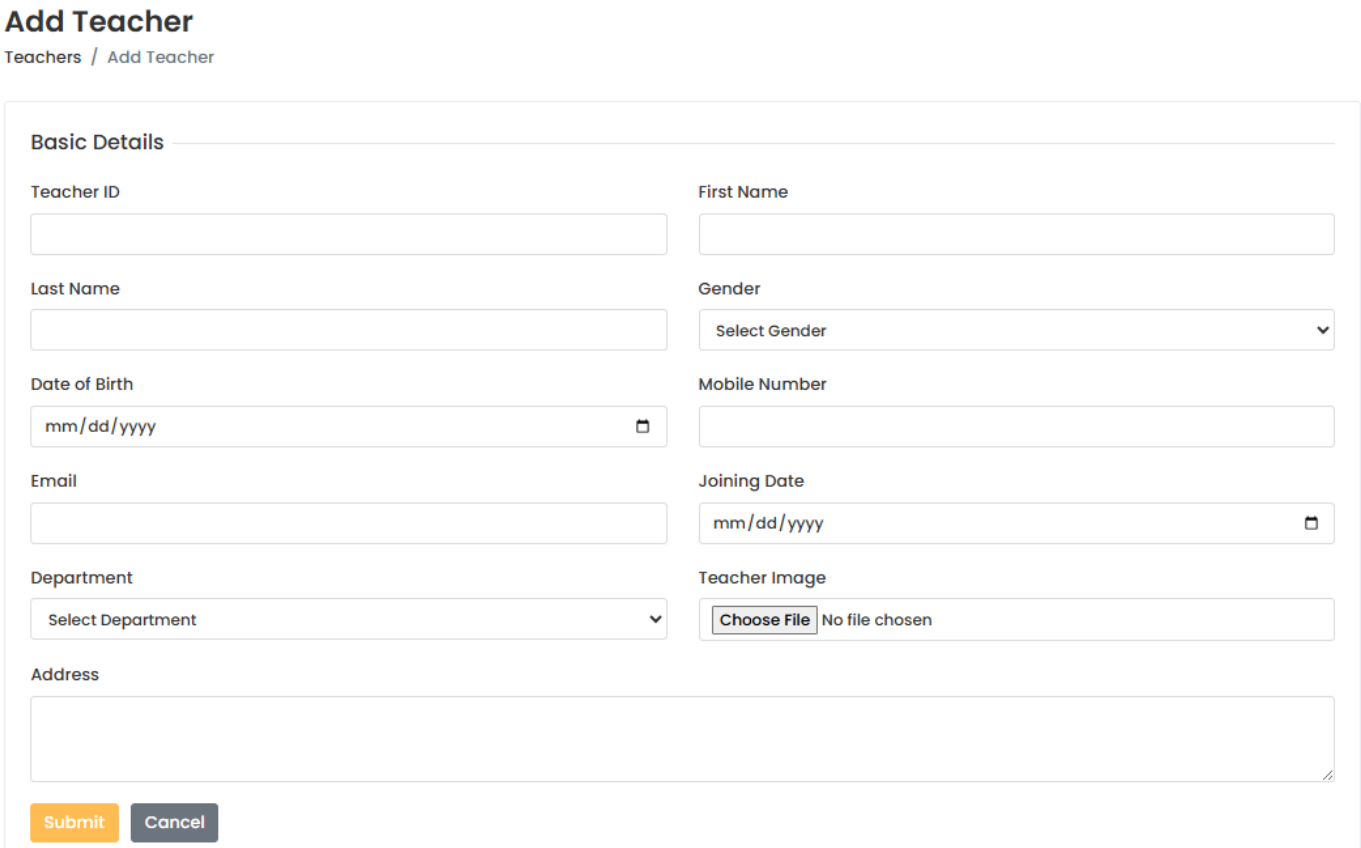
Section: 2

9.4 CRUD Enseignants

Liste des Enseignants :



Formulaire Ajout Enseignant — avec sélection du département :



9.5 Départements et Matières

Liste des Départements :

Departments

Add Department

#	Name	Description	Actions
1	Informatiquee	Ce departement vise l'AI et dev et cyber et data	Edit Delete
2	Computer Science	IT and Programming Department	Edit Delete
3	Data Science	Data analysis and AI	Edit Delete
4	Applied Mathematics	Math for engineering	Edit Delete
5	Life Sciences	Biology and health	Edit Delete

Liste des Matières — avec département et enseignant assignés :

Subjects

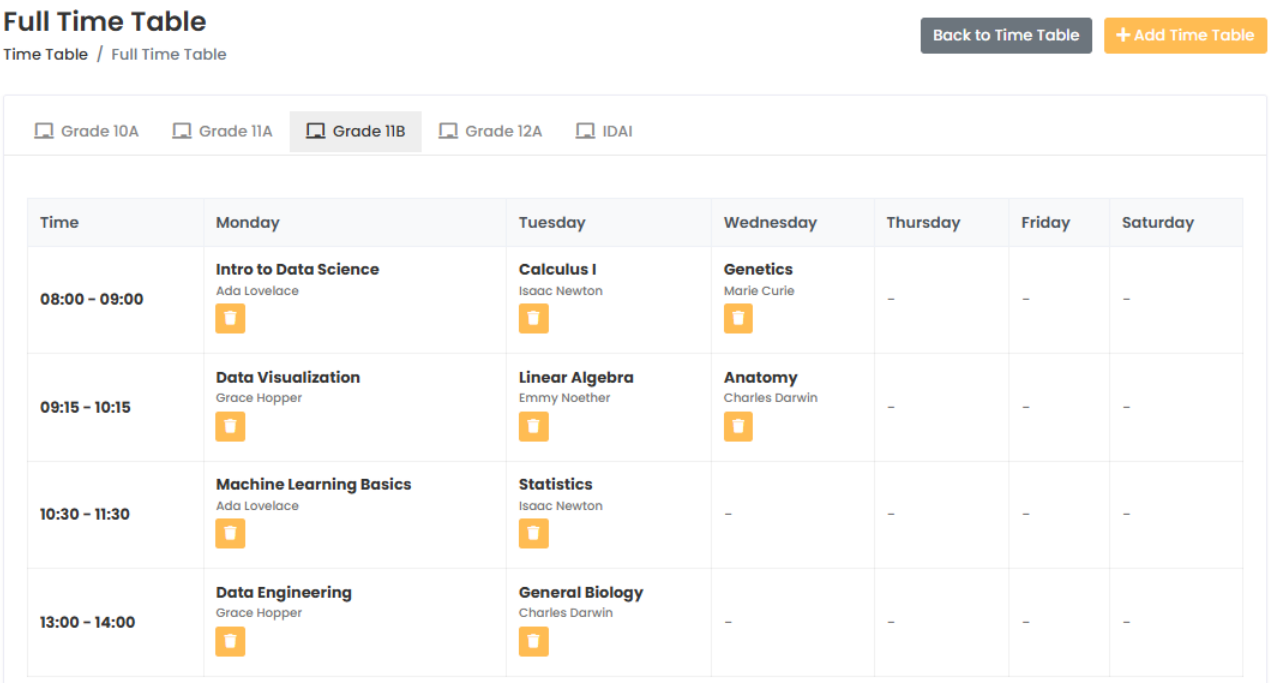
Dashboard / Subjects

+ Add Subject

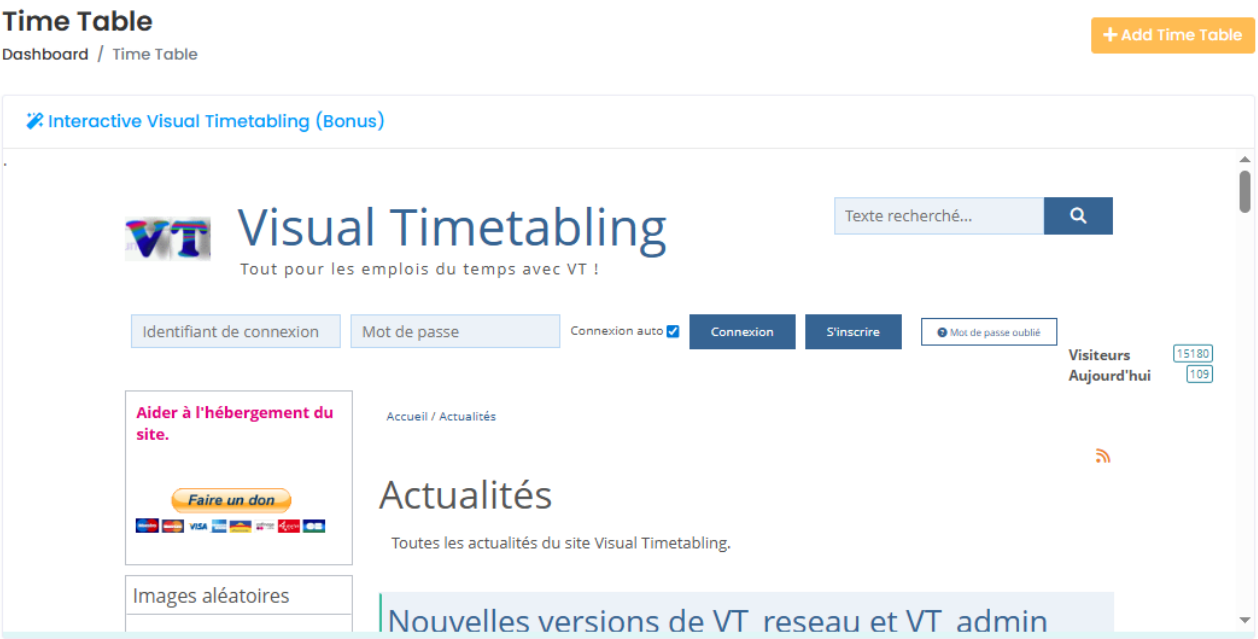
Subject Name	Subject Code	Department	Teacher	Action
Introduction to Pythone	CS101	Informatiquee	amer kk	
Intro to Data Science	DS101	Data Science	Ada Lovelace	
Data Visualization	DS102	Data Science	Grace Hopper	
Machine Learning Basics	DS201	Data Science	Ada Lovelace	
Data Engineering	DS202	Data Science	Grace Hopper	
Calculus I	MA101	Applied Mathematics	Isaac Newton	
Linear Algebra	MA102	Applied Mathematics	Emmy Noether	

9.6 Emploi du Temps :

Emploi du Temps — Vue tableau par jour et horaire :



Visual Timetabling — Intégration iframe (Bonus) :



9.7 Examens et Résultats :

Liste des Examens :

Exam List

Dashboard / Exams

+ Add Exam

Exam Name	Subject	Class	Date	Start Time	Action
amira	Introduction to Pythone (CS101)	AD	31 Mar 2026	23:51	Grades
ex2	Linear Algebra (MA102)	Grade 11A	02 Apr 2026	22:45	Grades
ex4	General Biology (LS101)	Grade 11B	02 Apr 2026	20:46	Grades
ex3	Genetics (LS102)	Grade 11B	13 Apr 2026	23:45	Grades
mid tearm	Calculus I (MA101)	Grade 11A	15 Apr 2026	22:00	Grades

Saisie des Notes — Note /20 avec mention calculée automatiquement :

Manage Results: ex2

Exams / Results

← Back to Exams

Add / Update Grade

Student

Select Student

Marks /20

e.g. 15.5

Save Grade

Current Grades

Student	Marks	Status (Grade)
Alex Smith	18.0 / 20	Mention (M)
Jordan Smith	10.0 / 20	Validated (V)
Sam Smith	8.0 / 20	Retake (RATT)
Casey Smith	3.0 / 20	Not Validated (NV)

9.8 Jours Fériés :

Calendrier des Jours Fériés

Holiday

Dashboard / Holiday

+ Add Holiday

Holiday Name	Date	Description	Action
Holiday 02 Apr	02 Apr 2026	Auto-added (every 6 days)	
VM DB 0001	04 Apr 2026	test	
Holiday 08 Apr	08 Apr 2026	Auto-added (every 6 days)	

10. Difficultés Rencontrées

Voici les problèmes concrets rencontrés pendant le développement, et comment ils ont été résolus.

10.1 Structure des Fichiers Statiques

Problème :

Les templates utilisaient {% static 'assets/css/style.css' %} mais les fichiers étaient placés directement dans static/ sans le sous-dossier assets/. Toutes les ressources retournaient 404 Not Found.

Solution :

Création du répertoire static/assets/ et déplacement de tous les fichiers CSS, JS et images à l'intérieur. Django cherchait exactement ce chemin car les templates avaient déjà été fournis avec cette structure.

10.2 AUTH_USER_MODEL et Ordre des Migrations

Problème :

AUTH_USER_MODEL = 'home_auth.CustomUser' doit être défini dans settings.py avant la toute première migration. Si on l'ajoute après, Django crée des conflits dans l'historique des migrations impossibles à résoudre sans réinitialisation.

Solution :

Suppression de db.sqlite3 et de tous les fichiers 0001_initial.py, puis reconstruction des migrations dans le bon ordre : home_auth en premier, ensuite department, teacher, subject, student, etc.

10.3 Décorateurs sur 30+ Vues

Problème :

Appliquer un contrôle d'accès cohérent sur plus de 30 vues réparties dans 8 applications différentes, sans dupliquer du code ni oublier de protéger une vue.

Solution : Création d'un fichier home_auth/decorators.py centralisé avec 3 décorateurs réutilisables. Il suffit d'importer le bon décorateur et de l'ajouter au-dessus de chaque vue — une seule ligne par vue.

10.4 Iframe Visual Timetabling

Problème :

Le site visual-timetabling.be bloque son affichage dans un <iframe> via l'en-tête HTTP X-Frame-Options: SAMEORIGIN. C'est une protection côté serveur tiers qu'on ne peut pas contourner.

L'implémentation dans notre code est correcte. Ce comportement est documenté dans le rapport comme une limitation externe, pas une erreur dans notre application.

12. Conclusion

Ce projet a été l'occasion de construire une application web Django complète de A à Z — de la configuration de l'environnement virtuel jusqu'au déploiement du système de rôles. Ce n'est pas juste un exercice académique : c'est un vrai système de gestion scolaire fonctionnel, avec des données réelles qui entrent et sortent d'une base de données.

Le fait de travailler avec 9 applications Django indépendantes a vraiment mis en pratique la notion de séparation des responsabilités. Chaque app a son propre models.py, views.py et urls.py — et elles communiquent entre elles uniquement via les clés étrangères et les imports.

Ce qui a été livré dans ce projet :

- 9 applications Django avec CRUD complet pour chaque entité
- Système de rôles RBAC à deux niveaux : décorateurs Python + conditions dans les templates
- 3 dashboards personnalisés selon le rôle de l'utilisateur connecté
- Calcul automatique des mentions sur 20 selon le système marocain
- Interface responsive avec le template PreSkool Bootstrap 4
- Intégration Visual Timetabling (bonus +5 pts)

— Fin du Rapport Technique —

PreSkool Academy | FST Tanger | 2025/2026